

A decorative graphic on the left side of the slide. It consists of a blue parallelogram and a light green parallelogram, both tilted at an angle. The blue shape is in the foreground, and the green shape is partially behind it. They are set against a dark blue background with faint, lighter blue diagonal stripes.

# Backpropagation & Gradient Flow



# Introduction

The aim of backpropagation is to optimise the weights and biases to minimise the errors or loss.

To understand the fundamentals of backpropagation and gradient flow in Convolution Neural Networks we need to understand how the loss is calculated.



# Softmax

- Normalises CNN outputs into probabilities from 0 to 1.
- Uses exponential function to push the largest value close to 1 and smallest value closer to 0.
- Sum of all outputs is always 100%, making it perfect for multi-class classification.

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$



# Cross-entropy Loss

- Almost always used with Softmax
- Uses the Softmax probabilities to determine how close those are to the original labels.
- Uses negative sign to output absolute value, since log of probabilities is always negative.
- When true label is 1, higher probabilities result in a smaller loss and lower probabilities result in a higher loss.

$$\text{CrossEntropy}(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$



# Softmax + Cross-entropy Example

*Logits* :  $z = [0.92, 1.0, 0.1]$

*Softmax* :  $\sigma(z) = [0.3962, 0.4292, 0.1745]$

$$\text{CrossEntropy}(y, \hat{y}) = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

*Loss* :  $L = 0.9257334971777009$

*Logits* :  $z = [3.57, 0.38, 1.06]$

*Softmax* :  $\sigma(z) = [0.8909, 0.0367, 0.0724]$

$$\text{CrossEntropy}(y, \hat{y}) = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

*Loss* :  $L = 0.115504985250951$



# The Chain Rule

- Used to find the derivative of composite functions.

In one simple line, the Chain Rule says:

To Differentiate  $f(g(x))$  with respect to  $x$ , take the derivative of  $f$  with respect to its input  $g(x)$ , and multiply by the derivative of  $g(x)$  with respect to  $x$ .

In Simpler terms:

The total rate of change is the derivative of the outside function multiplied by the derivative of the inside function.

$$\frac{d}{dx}[f(g(x))] = f'(g(x))g'(x)$$



# The Chain Rule in Backpropagation

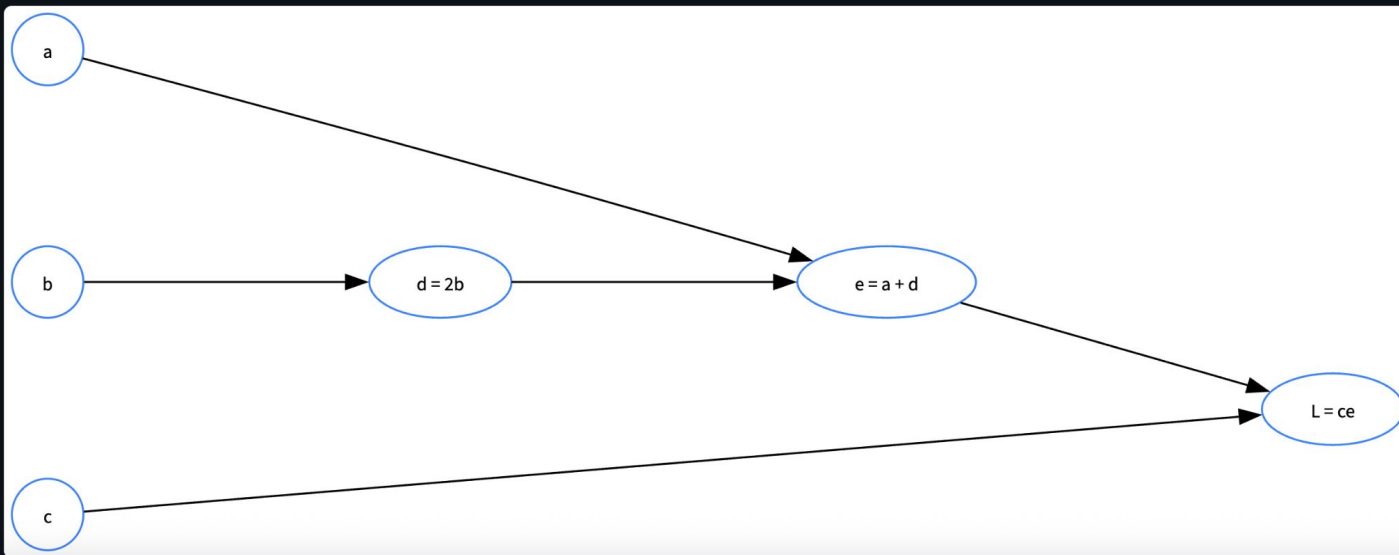
- Neural networks are essentially massive nested functions, the chain rule is the only way to propagate the gradient backward through each layer.
- Calculate how much a specific weight deep inside the network contributed to the final error at the output.
- Breaks Down complex problems into small, manageable pieces.
- Calculates the gradients which are used to update weights using optimisation algorithms such as Gradient Descent.

# The Chain Rule Example

$$L = ce : \quad \frac{\partial L}{\partial c} = e \quad , \quad \frac{\partial L}{\partial e} = c$$

$$L = a + d : \quad \frac{\partial e}{\partial a} = 1 \quad , \quad \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$





# The Chain Rule Example

## Backward Pass

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial a} = -2.0 \cdot 1.0 = -2.0$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial d} \cdot \frac{\partial d}{\partial b} = -2.0 \cdot 1.0 \cdot 2.0 = -4.0$$

$$\frac{\partial L}{\partial c} = \frac{\partial L}{\partial c} = 5.0$$

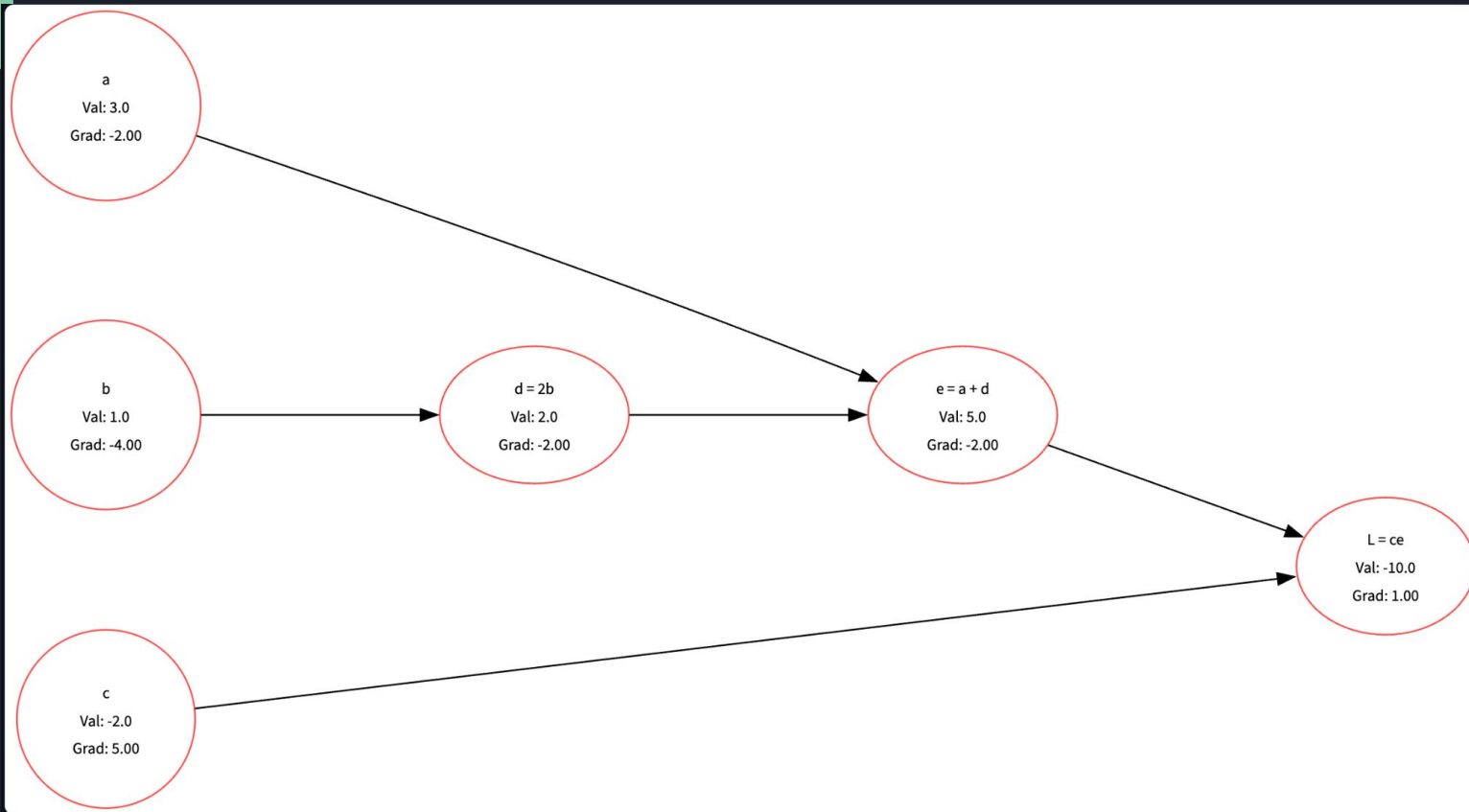
## Partial Derivatives

$$L = ce : \quad \frac{\partial L}{\partial c} = e \quad , \quad \frac{\partial L}{\partial e} = c$$

$$L = a + d : \quad \frac{\partial e}{\partial a} = 1 \quad , \quad \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$

# The Chain Rule Example





# Vanishing Gradient Problem

- Gradients become extremely small as they are multiplied through many layers.
- Layers closest to the input receive almost no signal to change. The model stops learning, and the weights in early layers remain stagnant.

## Common Fixes:

1. Using ReLU as Activation function
2. Skip Connections to bypass layers



# Exploding Gradient Problem

- Opposite of vanishing gradients. Gradients grow exponentially large during backpropagation.
- Weight updates are massive, causing the model to miss the optimal solution. The training becomes unstable.

## Common Fixes:

1. Gradient Clipping, setting a maximum threshold for gradients
2. Weight Initialising, carefully choosing initial weights



# Computational Graphs

- Computational graphs are organized as a series of Nodes connected by Edges, where data flows in a specific direction without looping back on itself.
- Each node in the graph represents a specific mathematical operation or a function such as ReLU or Softmax.
- Edges represent the flow of data. During the forward pass, edges carry activation values. During the backward pass, these same edges facilitate the movement of gradient values in the opposite direction.



# Advantages of Computational Graphs

- The graph knows exactly which edges connect which operations, it can automatically apply the Chain Rule to compute gradients.
- Maps out which operations depend on each other, deep learning frameworks can identify independent branches. These branches can be executed simultaneously on different GPU cores , speeding up training time.
- Optimisation before training, the system can look at the graph and remove edges or operations that don't contribute to the final loss, or fuse multiple nodes into one to reduce memory overhead.



# Advantages of Computational Graphs

- The graph knows exactly which edges connect which operations, it can automatically apply the Chain Rule to compute gradients.
- Maps out which operations depend on each other, deep learning frameworks can identify independent branches. These branches can be executed simultaneously on different GPU cores , speeding up training time.
- Optimisation before training, the system can look at the graph and remove edges or operations that don't contribute to the final loss, or fuse multiple nodes into one to reduce memory overhead.



# Updating the Weights

- Gradient descent is used to take the calculated gradients and uses them to adjust the weights to minimize the model's total error.
- The learning rate acts as the step size:
  - If set too low, the model would take a long time to train.
  - If set too high, the model will overshoot the target and fail to learn.

$$W_{new} = W_{old} - \eta \cdot \frac{dL}{dW}$$



